

Institut Teknologi Del

Fakultas Informatika dan Teknik Elektro

Program Studi Sarjana Sistem Informasi • Sarjana Informatika

MODUL PRAKTIKUM

MINGGU 04

KONSEP DASAR ARSITEKTUR APLIKASI WEB KONTEMPORER: DECOUPLED MULTI-TIER, DYNAMIC CLIENT-SIDE RENDERING (CSR), DAN ANALISIS KINERJA WEB

Mata Kuliah	: Pemrograman dan Pengujian Web (12S3101)	Dosen Pengampu	: Chandro Pardede, S.Kom., M.Sc.
Keterkaitan Materi	: Melanjutkan & Mentransformasi Tugas Minggu 3	Alokasi Waktu	: 170 Menit Terbimbing + 360 Menit Mandiri
Tahun Akademik	: Semester Genap 2025/2026	Format Penugasan	: Git Repository & GitHub Pages Live

I. Capaian Pembelajaran & Tujuan Praktikum

Capaian Pembelajaran Khusus (Sub-CPMK) SubCPMK12-02-09-03: Mampu menjabarkan konsep dasar arsitektur aplikasi web kontemporer dengan tepat. [C2, A2, P2]

Indikator Ketercapaian Terukur:

- Pemodelan Arsitektur Multi-Tier:** Mahasiswa mampu mendekomposisi aplikasi web menjadi *Presentation Tier*, *Application/API Logic Tier*, dan *Data Storage Tier*, serta memodelkannya ke dalam Diagram C4 Container Model.
- Analisis Paradigma Rendering:** Mengidentifikasi perbedaan arsitektural dan trade-offs antara *Monolith vs Microservices*, *Server-Side Rendering (SSR)*, *Client-Side Rendering (CSR)*, dan *Jamstack*.
- Transformasi Proyek Minggu 3 ke Dynamic CSR:** Menghapus konten kartu HTML yang sebelumnya ditulis secara statis (*hardcoded*) dan menggantikannya dengan pemuatan data dinamis menggunakan JavaScript modern (ES6+ `fetch()` dan `async/await`).
- Desain Data Provider Modular JSON:** Merancang struktur data mandiri (`projects.json`, `services.json`, `profile.json`) yang bertindak sebagai *decoupled mock RESTful data layer*.
- Manajemen Siklus Status Antarmuka (UI States):** Mengimplementasikan penanganan visual status antarmuka komprehensif: *Loading State* (Skeleton/Spinner), *Success Render State*, *Empty State*, dan *Error Fallback Alert*.
- Universal Dynamic Modal Component:** Mengonfigurasi satu komponen modal tunggal Bootstrap 5 yang dapat menyajikan rincian proyek secara dinamis berdasarkan data-ID tanpa duplikasi elemen HTML.

7. **Decoupled Asynchronous REST Form Dispatching:** Merefaktor formulir layanan Week 3 menjadi mekanisme pengiriman asinkron (AJAX/Fetch POST) dengan serialisasi JSON DTO dan umpan balik visual Toast tanpa memicu *full page reload*.
8. **Manajemen State Lokal Sisi Klien:** Menyimpan riwayat pemesanan layanan ke dalam `localStorage` / `sessionStorage` sebagai lapisan persistensi terdistribusi di sisi klien.
9. **Analisis Caching & Profil Kinerja (RFC 9111):** Mengukur *Time to First Byte* (TTFB), *First Contentful Paint* (FCP), hierarki *Waterfall*, dan efisiensi header HTTP Caching (`Cache-Control`, `ETag`, `304 Not Modified`) melalui Browser DevTools.
10. **Keamanan Sisi Klien Lapis Pertama:** Menerapkan sanitasi masukan guna mencegah kerentanan *DOM-based Cross-Site Scripting* (XSS) dan merancang direktif *Content Security Policy* (CSP).

II. Alat dan Bahan (Prerequisites)

- **Code Editor:** Visual Studio Code dengan ekstensi *Live Server*, *Prettier*, *ESLint*, dan *Thunder Client / REST Client*.
- **Web Browser Modern:** Google Chrome atau Microsoft Edge dengan DevTools aktif (Tab Network, Console, Application, Performance).
- **Basis Kode Tugas Minggu 3:** Repositori lokal hasil pengerjaan minggu 3 (`ppw-2026-week3-[NIM]`) berbasis Bootstrap 5.3.
- **Version Control & Hosting:** Git CLI terpasang dan akun GitHub aktif untuk pembuatan cabang (*branching*) serta deployment GitHub Pages.
- **Diagramming Tool:** Ekstensi Mermaid.js di Markdown atau Draw.io / Excalidraw untuk menggambar diagram arsitektur.

III. Landasan Teori dan Konsep Kunci

3.1 Hakikat dan Evolusi Arsitektur Perangkat Lunak Web

Arsitektur perangkat lunak web mendefinisikan struktur organisasi sistem, komponen logika, antarmuka komunikasi antar-layanan, serta batasan operasionalnya. Perkembangan arsitektur web bergerak dari paradigma monolitik terpusat menuju sistem yang terdistribusi dan terpisah (*decoupled*):

Evolusi Paradigma Arsitektur Web

1. **Era Web 1.0 (Static MPA):** Browser → Meminta file `.html` → Web Server membaca disk → Mengirimkan HTML statis murni. Navigasi selalu memicu pemuatan ulang halaman penuh (*full page reload*).
2. **Era Web 2.0 (Server-Side Dynamic - Monolithic SSR):** Browser → HTTP GET/POST → Server Aplikasi (PHP/Java/ASP) mengeksekusi logika → Query Database → Menggabungkan template dan data di server → Mengirimkan HTML utuh.
3. **Era Web Kontemporer (Decoupled, Jamstack, SPA & Microservices):** Frontend di-hosting di CDN Edge → Browser memuat HTML shell mini → Eksekusi JavaScript memanggil data JSON via RESTful API secara asinkron → DOM dirakit langsung di browser pengguna.

3.2 Pola Arsitektur Berlapis (Multi-Tier Architecture)

Aplikasi web kontemporer membagi tanggung jawab fungsional ke dalam lapisan terpisah (*Separation of Concerns*):

- **Presentation Tier (Client / Browser):** Mengelola antarmuka pengguna (HTML5, Bootstrap 5 CSS, Custom Styles) dan logika kontrol tampilan berbasis JavaScript ES6+. Bertanggung jawab terhadap responsivitas dan interaktivitas instan.
- **Application / Service Logic Tier (API Gateway / Backend Services):** Mengontrol aturan bisnis, validasi masukan, otorisasi, dan transformasi data melalui kontrak API terstandarisasi (REST JSON / GraphQL). *Pada modul ini disimulasikan menggunakan decoupled JSON provider dan mock REST endpoint.*
- **Data Storage Tier:** Menyimpan catatan data secara persisten dan konsisten (basis data relasional, NoSQL, atau cloud storage).

3.3 Paradigma Rendering: Komparasi SSR vs CSR vs Jamstack

Parameter Evaluasi	Server-Side Rendering (SSR)	Client-Side Rendering (CSR)	Jamstack / Decoupled Static
Perakitan Elemen DOM	Di Server Aplikasi per request	Di Browser pengguna via JavaScript	Saat Build-Time & Hydrated via API
Beban Komputasi Server	Tinggi (server merender HTML)	Sangat Rendah (hanya transfer data)	Minimal (Aset disajikan CDN Edge)
Time to First Byte (TTFB)	Menengah hingga Lambat	Sangat Cepat (HTML shell mini)	Sangat Cepat (< 50ms dari cache CDN)
Interaktivitas Pengguna	Kaku (Full reload tiap navigasi)	Sangat Mulus (Navigasi instan)	Sangat Mulus & Reaktif
Infrastruktur Hosting	Server runtime aktif 24/7	Static CDN (GitHub Pages, Vercel)	Static CDN + Serverless / API

3.4 Request-Response Asinkron Menggunakan Fetch API & UI States

Fetch API berbasis Promise memungkinkan peramban meminta data dari penyedia layanan secara asinkron tanpa memblokir thread rendering utama. Pengembang wajib menangani empat status antarmuka secara visual:

```
// Pola Pengambilan Data Asinkron Modern dengan Defensive Error Handling
async function fetchPortfolioData() {
  try {
    const response = await fetch('./data/projects.json');
    if (!response.ok) {
      throw new Error(`HTTP Error ${response.status}: ${response.statusText}`);
    }
    const data = await response.json();
    return data;
  } catch (err) {
    console.error('[API Network Error]:', err);
    throw err;
  }
}
```

Defense in Depth: Bahaya DOM XSS pada Arsitektur CSR Menyuntikkan data JSON mentah langsung ke dalam `.innerHTML` rentan terhadap celah keamanan *Cross-Site Scripting*

(XSS). Selalu lakukan sanitasi string atau gunakan `textContent` saat merender nilai dinamis masukan pengguna.

3.5 Strategi Caching Berjenjang & Standar HTTP RFC 9111

Sistem caching mengeliminasi latensi transmisi jaringan dengan menyimpan salinan representasi berkas:

- **Cache-Control: max-age=3600:** Menginstruksikan peramban bahwa salinan lokal valid selama 1 jam.
- **Cache-Control: no-cache:** Peramban boleh menyimpan berkas, namun wajib memvalidasi ke server sebelum menggunakannya.
- **ETag & Status 304 Not Modified:** Jika sidik jari (hash) berkas tidak berubah, server mengirim kode 304 dengan body kosong, menghemat pemakaian bandwidth hingga 95%.

IV. Kegiatan Praktikum Terbimbing (Hands-on Labs)

Lab 1: Dekomposisi Arsitektur & Pemodelan C4 Container Diagram (40 Menit)

Mahasiswa membedah arsitektur monolitik statis dari Week 3 menjadi arsitektur multi-tier kontemporer yang terpisah, serta menyusun struktur folder kerja modular:

```
# Struktur Direktori Terstandarisasi Week 4:
ppw-2026-week4-[NIM]/
├── index.html                # Shell HTML5 & Bootstrap 5 bersih tanpa
hardcoded cards
├── css/
│   └── custom-style.css     # Custom styles, theming & CSS variables
dari Week 3
├── data/
│   ├── profile.json         # Biodata pengembang & statistik performa
│   └── projects.json        # Data terstruktur koleksi portofolio
proyek
├── services.json            # Katalog paket layanan, fitur, dan tarif
├── js/
│   ├── api-service.js       # Data Access Layer: Pemanggilan HTTP
Fetch & Error Handling
│   └── app.js               # Presentation Layer: Kontrol DOM, Dynamic
Rendering & Events
└── README.md                # Dokumentasi diagram arsitektur &
komparasi kinerja
```

Lab 2: Implementasi Dynamic Client-Side Rendering & Universal Modal (45 Menit)

Pada lab ini, mahasiswa menyusun berkas `data/projects.json` dan membangun modul `api-service.js` serta `app.js` untuk merender kartu portofolio dan mengoperasikan **Universal Dynamic Modal**:

```

<!-- Universal Dynamic Modal: 1 Modal Tunggal di index.html untuk Seluruh
Proyek -->
<div class="modal fade" id="universalProjectModal" tabindex="-1">
  <div class="modal-dialog modal-lg modal-dialog-centered">
    <div class="modal-content border-0 shadow-lg">
      <div class="modal-header bg-primary text-white">
        <h5 class="modal-title fw-bold" id="projectModalTitle">Judul
Proyek</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-
dismiss="modal"></button>
      </div>
      <div class="modal-body p-4" id="projectModalBody">
        <!-- Konten diinjeksi secara dinamis via JavaScript sesuai project-
ID yang diklik -->
      </div>
    </div>
  </div>
</div>
// app.js: Logika Pembukaan Modal Universal Dinamis Berbasis ID
openProjectModal(projectId) {
  const proj = this.state.projects.find(p => p.id === projectId);
  if (!proj) return;

  document.getElementById('projectModalTitle').textContent = proj.title;
  document.getElementById('projectModalBody').innerHTML = `
    
    <p class="text-secondary">${this.escapeHTML(proj.description)}</p>
    <div class="badge bg-primary px-3 py-2">${proj.category}</div>
  `;

  const modalEl = document.getElementById('universalProjectModal');
  bootstrap.Modal.getOrCreateInstance(modalEl).show();
}

```

Lab 3: Decoupled REST Form Dispatch & DevTools Profiling (45 Menit)

Merefaktor formulir pemesanan layanan Week 3 menjadi pengiriman asinkron (AJAX/Fetch POST) dengan serialisasi DTO JSON dan feedback Bootstrap Toast:

```

// app.js: Form Submit Asinkron Tanpa Reload Halaman Penuh
form.addEventListener('submit', async (e) => {
  e.preventDefault(); // Menghentikan perilaku reload standar

  const formData = new FormData(form);
  const payload = Object.fromEntries(formData.entries());

  const submitBtn = form.querySelector('button[type="submit"]');
  submitBtn.disabled = true;
  submitBtn.innerHTML = '<span class="spinner-border spinner-border-
sm"></span> Mengirim...';

  try {
    const result = await ApiService.submitServiceOrder(payload);
    this.saveOrderToLocalStorage(payload);
    this.showToastNotification('Sukses!', 'Permintaan layanan berhasil
diproses oleh API.');
```

```

        submitBtn.innerHTML = 'Kirim Permintaan';
    }
});

```

V. Tugas Mandiri Mahasiswa (Individual Assignment)

Judul Tugas Mandiri Minggu 4: Refactoring Arsitektural Personal Portfolio & Service Portal: Implementasi Decoupled Multi-Tier, Dynamic Client-Side Rendering (CSR), dan Network Performance Profiling (Melanjutkan Tugas Minggu 3)

5.1 Keterkaitan Langsung dengan Tugas Minggu 3 (Continuity)

Pada Praktikum Minggu 3, mahasiswa telah berhasil membangun portofolio dengan Bootstrap 5. Namun, seluruh data kartu, teks modal, dan katalog layanan masih berstatus **monolitik statis (hardcoded di dalam berkas `index.html`)** serta formulir belum terhubung dengan arsitektur data terpisah.

Pada Praktikum Minggu 4 ini, **mahasiswa diwajibkan melanjutkan dan mentransformasikan repositori Minggu 3 tersebut** menjadi aplikasi web berarsitektur kontemporer yang *decoupled*, di mana seluruh konten diinjeksi secara asinkron dari sumber data JSON modular, menggunakan Universal Dynamic Modal, mengimplementasikan pengiriman form RESTful, dan teruji performa jaringannya melalui DevTools.

5.2 Spesifikasi Teknis Wajib (Requirements Checklist)

Area Evaluasi	Bobot	Spesifikasi & Standar yang Wajib Terpenuhi
1. Pemodelan Arsitektur Web	15%	Dokumentasi <code>README.md</code> memuat Diagram Arsitektur Sistem C4 Container Model (Mermaid / visual grafis) yang memetakan Client, Static Server, CDN, JSON Providers, dan REST API; disertai narasi ilmiah pemisahan minat (<i>Separation of Concerns</i>).
2. Dekomposisi Data Layer JSON	20%	Seluruh data dipindahkan ke direktori <code>/data:</code> <code>projects.json</code> (minimal 4 proyek lengkap dengan metrics, tags, image, link), <code>services.json</code> (minimal 3 paket layanan), dan <code>profile.json</code> (data diri mahasiswa). Format JSON valid dan terstruktur rapi.
3. Dynamic CSR & UI States	25%	Shell <code>index.html</code> bersih dari kartu hardcoded; data dimuat via <code>api-service.js</code> & <code>app.js</code> berbasis <code>async/await</code> ; 4 UI States terkelola sempurna: <i>Loading</i> (spinner/skeleton), <i>Success</i> (render kartu), <i>Empty</i> (filter kosong), dan <i>Error</i> (alert peringatan defensif); Filter Kategori berfungsi instan.
4. Universal Dynamic Modal	15%	Tepat 1 elemen modal universal di <code>index.html</code> ; tombol kartu memicu injeksi rincian proyek secara dinamis berdasarkan data-ID tanpa duplikasi elemen HTML; integrasi Bootstrap 5 Modal API mulus dan aman dari XSS.
5. Decoupled Form REST & State	15%	Formulir layanan Week 3 dikirim secara asinkron murni via HTTP POST (no full page reload); status tombol submit

Area Evaluasi	Bobot	Spesifikasi & Standar yang Wajib Terpenuhi
6. Network Profiling DevTools	10%	responsif; notifikasi feedback visual Bootstrap Toast interaktif; data pesanan disimpan secara persisten di <code>localStorage</code> dan ditampilkan di UI badge. Melakukan pengujian profil jaringan tab Network DevTools: tabel perbandingan Cold Load vs Warm Load, analisis status HTTP 304 Not Modified, TTFB, dan screenshot waterfall dilampirkan pada <code>README.md</code>.

VI. Instruksi Pengelolaan Git & Pengumpulan Tugas

- Manajemen Branch / Repositori Git:**
- # Masuk ke folder proyek minggu 3, buat cabang baru:
- `git checkout -b week4-architecture`
- # Lakukan refactoring arsitektural, pengujian jaringan, dan komit:
- `git add .`
- `git commit -m "feat(week4): decouple architecture to json data providers and async CSR"`
`git push -u origin week4-architecture`
- Aktifkan GitHub Pages:** Buka repositori di GitHub → Settings → Pages → Source: Branch `week4-architecture` (atau `main`) → Save.
- Dokumentasi README.md:** Wajib memuat Diagram Arsitektur C4, Tabel Komparasi "Sebelum vs Sesudah Refactoring", Tabel Pengukuran DevTools, dan Tautan Live Deployment.
- Batas Waktu Pengumpulan:** Senin, 16 Februari 2026, Pukul 23.59 WIB.

VII. Rubrik Penilaian Praktikum Analitik

Kriteria Evaluasi	Bobot	Sangat Baik (85 - 100)	Baik (70 - 84)	Cukup / Kurang (< 70)
Pemodelan Arsitektur C4 & SoC	20%	Diagram C4 Container sangat jelas memetakan Client, Static Server, CDN, JSON Providers, dan REST API; narasi ilmiah SoC dan komparasi arsitektur sangat mendalam.	Diagram arsitektur tersaji lengkap; narasi komparasi cukup baik; seluruh layer utama telah terpetakan dengan benar.	Diagram tidak lengkap atau hanya alur flowchart biasa; narasi komparasi dangkal atau tidak ada diagram.
Dynamic CSR & UI States	25%	HTML bersih dari hardcoded cards; JSON valid; rendering dinamis <code>async/await</code> mulus; 4 UI states (loading, success, empty, error) terkelola	Data JSON terpisah dan dirender dinamis; UI states diimplementasikan namun handling error atau loading state masih kurang rapi.	Data masih ditulis manual secara hardcoded di HTML atau JavaScript gagal merender data ke antarmuka.

Kriteria Evaluasi	Bobot	Sangat Baik (85 - 100)	Baik (70 - 84)	Cukup / Kurang (< 70)
Universal Dynamic Modal	15%	sempurna disertai filter kategori instan. Tepat 1 elemen modal universal di HTML; injeksi data dinamis berbasis ID berjalan presisi; terhubung sempurna dengan Bootstrap 5 API; aman dari serangan DOM XSS.	Modal universal berfungsi baik menampilkan rincian proyek, namun terdapat sedikit inkonsistensi styling pada modal body.	Masih menggunakan modal statis terpisah di HTML, atau pemanggilan modal menghasilkan error eksekusi konsol.
Decoupled Form & Local State	20%	Form submit asinkron murni (no reload); payload JSON terstruktur dikirim via HTTP POST; feedback visual Toast dinamis; persistensi ke localStorage reaktif.	Form submit asinkron dengan payload JSON dan feedback visual baik; data tersimpan di storage namun state badge order belum reaktif.	Form masih memicu full page reload standar atau gagal mengirim data secara asinkron.
Profiling DevTools & Git	20%	Tabel pengukuran DevTools (TTFB, Caching 304, Cold/Warm Load) lengkap disertai screenshot valid; Git commit deskriptif; live demo GitHub Pages berfungsi tanpa error.	Pengukuran DevTools disajikan dengan baik; Git commit rapi; GitHub Pages aktif dengan minor catatan pada asset loading.	Tidak menyajikan data profiling DevTools; repositori tidak terkelola dengan baik; link live demo menghasilkan 404 Not Found.

Mengetahui,
Koordinator KBK Rekayasa Perangkat Lunak
 Institut Teknologi Del

(Tim Pengajar KBK RPL-SI)
 NIDN. 0123456789

Sitoluama, 09 Februari 2026
Dosen Pengampu Mata Kuliah
 Pemrograman dan Pengujian Web

Chandro Pardede, S.Kom., M.Sc.
 NIDN. 0102030405